

Dynamic Programming

Fibonacci Numbers

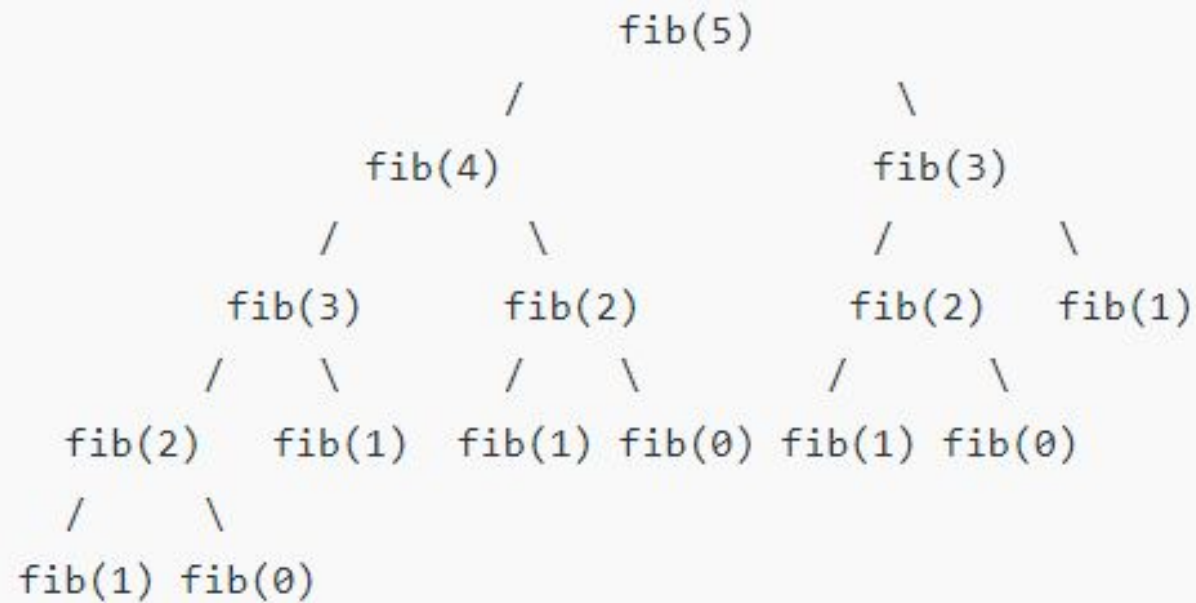
- The series: 0 1 1 2 3 5 8 13 21...
- $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$
- A simple recursive implementation

```
int fib(int n)
{
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```

Fibonacci Numbers

- Any drawback for the previous solution?

Fibonacci Numbers



Complexity (Recursive Solution)

- The highest depth of the recursion tree will be n .
- For a complete binary tree of height h , have $2^h - 1$ node.

It is an exponential runtime

Memoization

- Store the results of the subproblems (creating a table or memo)
- Instead of computing the subproblems every time, if it is calculated before, lookup in the memo and return

memo	-1	-1	-1	-1	-1	-1	-1	-1
------	----	----	----	----	----	----	----	----

```
int fib(int n)
{
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```



```
int fib(int n)
{
    if(memo[n] != -1)
        return memo[n]
    if (n <= 1){
        return memo[n] = n;
    }
    return memo[n] = fib(n - 1) + fib(n - 2);
}
```

Memoization (Top Down Approach)

memo	-1	-1	-1	-1	-1	-1	-1
set	0	0	0	0	0	0	0

```
int fib(int n)
{
    if(set[n])
        return memo[n]
    set[n] = 1;
    if (n <= 1){
        return memo[n] = n;
    }
    else
        return memo[n] = fib(n - 1) + fib(n - 2);
}
```

Bottom up approach (Tabulation)

memo	-1	-1	-1	-1	-1	-1	-1
------	----	----	----	----	----	----	----

```
int fib(int n)
{
    if (n <= 1){
        return memo[n];
    }
    for(int i=2; i<=n; i++)
        memo[i] = memo[i-1]+ memo[i - 2];
    return memo[n];
}
```


Bottom up approach (Tabulation)

memo	0	1	-1	-1	-1	-1	-1	-1
------	---	---	----	----	----	----	----	----

```
int fib(int n)
{
    if (n <= 1){
        return = memo[n];
    }
    for(int i=2; i<=n; i++)
        memo[n] = memo[n-1]+ memo[n - 2];
    return memo[n];
}
```

Bottom up approach (Tabulation)

memo	0	1	-1	-1	-1	-1	-1	-1
------	---	---	----	----	----	----	----	----

```
memo[0] = 0;
memo[1] = 1;

int fib(int n)
{
    if (n <= 1){
        return = memo[n];
    }
    for(int i=2; i<=n; i++)
        memo[n] = memo[n-1]+ memo[n - 2];
    return memo[n];
}
```

Runtime Complexity:

- Fill-up the DP table by calculating each value only once

Total Time needed:

N * Time required to combine the pre-calculated results

For Fibonacci Problem: Combining takes constant time.

So overall Complexity: $O(N)$

Two core characteristics of DP:

- **Overlapping Subproblems:** The same smaller problems keep popping up over and over again.
- **Optimal Substructure:** The optimal solution to the big problem can be constructed easily from the optimal solutions of its subproblems.

For Fibonacci problem:

- **Overlapping Subproblems:** For Fibonacci number generation problem:

To find $F(5)$, we need $F(4)$ and $F(3)$.

To find $F(4)$, we need $F(3)$ and $F(2)$.

We can see $f(3)$ is being repeated (overlapped).

- **Optimal Substructure:**

If we know the optimal solution for $f(3)$ and $f(4)$, this can help us to find the optimal solution for $f(5)$.

Friends Pairing Problem

Given n friends, each one can remain single or can be paired up with some other friend.

Each friend can be paired only once.

Find out the total number of ways in which friends can remain single or can be paired up.

(How many unique configurations of singles and pairs can you form from a group of n people?)

Friends Pairing Problem

- Input : $n = 3$
- Output : 4
- Explanation:
 - $\{1\}, \{2\}, \{3\}$
 - $\{1, 2\}, \{3\}$
 - $\{2\}, \{1, 3\}$
 - $\{1\}, \{2, 3\}$

Friends Pairing Problem

- Input : $n = 4$
- Output : 10
- Explanation:

When 4 remains alone: • $\{1\}, \{2\}, \{3\} - \{4\}$ • $\{1, 2\}, \{3\} - \{4\}$ • $\{2\}, \{1, 3\} - \{4\}$ • $\{1\}, \{2, 3\} - \{4\}$	When 4 pairs with someone: (i.e 1) • $\{2\}, \{3\} - \{4, 1\}$ • $\{2, 3\} - \{4, 1\}$ How many options person 4 have, to be paired with someone? (3)
--	---

Friends Pairing Problem

- Find smaller overlapping sub problems

Friends Pairing Problem

- Total number of ways to make friends with n people:
 - N^{th} person may decide to remain single
 - Or
 - N^{th} person may decide to pair up with remaining $(n-1)$ person

For 1st choice: Solution is same as solution for $(n-1)$ person

For 2nd choice: N^{th} person will have $(n-1)$ choice to pair up
For each choice, there are remaining $(n-2)$ people

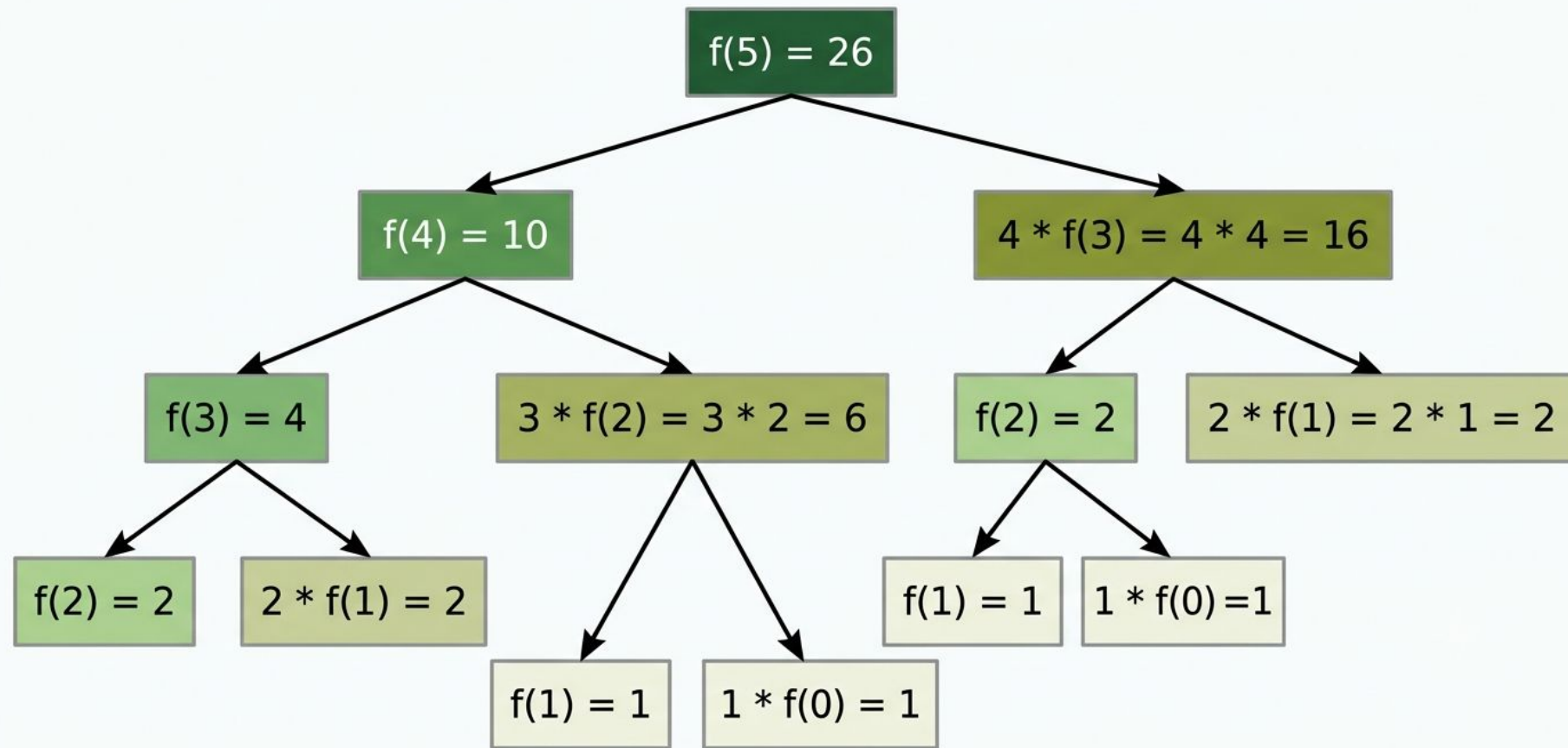
Friends Pairing Problem

Recurrence Relation:

- $f(n) = f(n - 1) + (n - 1) * f(n - 2)$

Friends Pairing Problem

Recursive Tree for Friends Pairing Problem (n=5)



Complexity

- Similar to Fibonacci series

Tiling Problem (V1)

- We have 2×1 unlimited supply of tiles
- Have to cover a $2 \times n$ sized board without overlapping
- Calculate total ways we can tile

Tiling Problem (V1)

- If $n = 3$

1	2	3
1	2	3

1	1	3
2	2	3

1	2	2
1	3	3

Tiling Problem (V1)

- If $n = 4$

1	2	3	4
1	2	3	4

1	1	3	3
2	2	4	4

1	1	3	4
2	2	3	4

1	2	2	4
1	3	3	4

1	2	3	3
1	2	4	4

Tiling Problem (V1)

- Recurrence relation?

Tiling Problem (V1)

First tile placed vertically:

Remaining size to tile?

First tile placed horizontally: (Another tile must be placed horizontally below):

Remaining size to tile ?

Tiling Problem (V1)

First tile placed vertically:

Remaining size to tile is $n-1$

First tile placed horizontally: (Another tile must be placed horizontally below):

Remaining size to tile: $n-2$

- $f(n) = f(n-1) + f(n-2)$

Performance Comparison

```
10 19
100 199
1000 1999
5000 9999
10000 19999
```

With DP table

```
10 177
20 21891
30 2692537
40 331160281
45 3672623805
```

Without DP table